

Monadic Metaprogramming with Bananas, Lenses and Envelopes

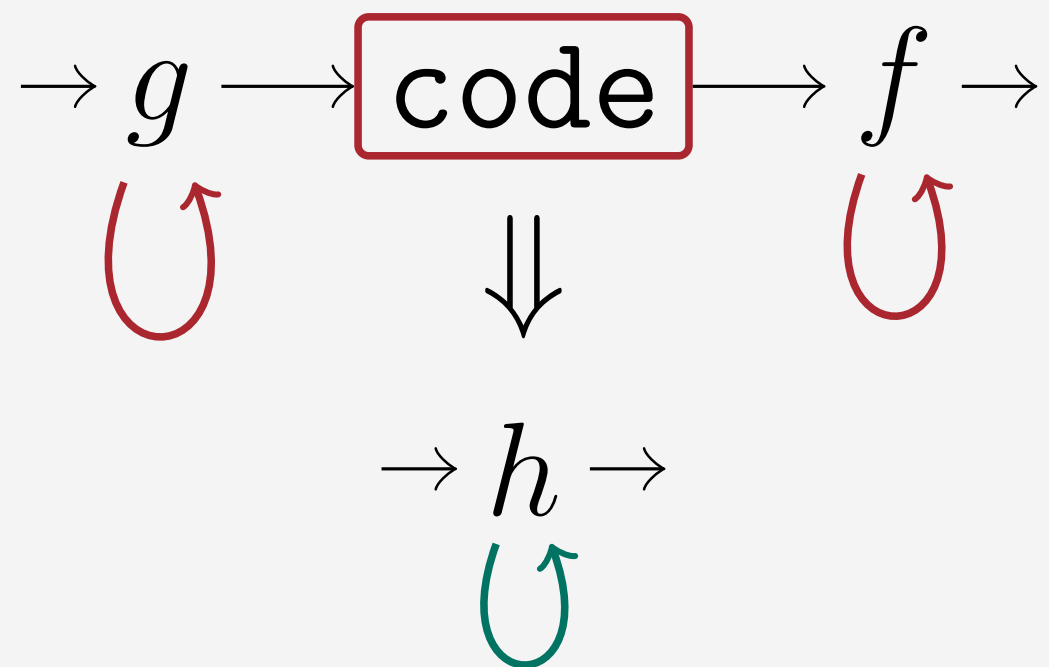
A Fusion Framework

李 張帆 亀山 幸義 (筑波大学)

研究概要

研究目標: コード生成器や解析器を組み合わせたメタプログラムをモジュラーかつ効率よく構成したい

着想: コード生成器(g)や解析器(f)をFold/Unfold/Hyloの形で記述し, フュージョンで高速化



✓ モジュール性

複雑なメタプログラムを小規模な部品に分解し, 保守性や再利用性を向上

✓ パフォーマンス

- 複数の再帰関数を一つに変換
- 中間データ構造 (incl. `code`) を消去

① プログラムのデータ構造は?

既存研究: 代数的データ型 [Takano1995]
本研究: 束縛機構を持つ構造 (PHOAS)
`data Exp v x = Var v | App x x
| Lam (v -> x) | Let x (v -> x)`

③ フュージョン規則の拡張が必要?

サポートしたい操作: 抽象解釈, 部分評価

- 新たなフュージョン規則を提案
- 整数演算の部分評価に対して実験を行った (例)
- フュージョンによる平均効率向上率 ≈ 4.72

② 既存研究の適用範囲は?

既存研究: リスト操作 (e.g., map, filter)

- 既存研究が扱えるメタプログラムを考察
- 数論変換の最適化に対して実験を行った (例)
- フュージョンによる平均効率向上率 ≈ 14.4

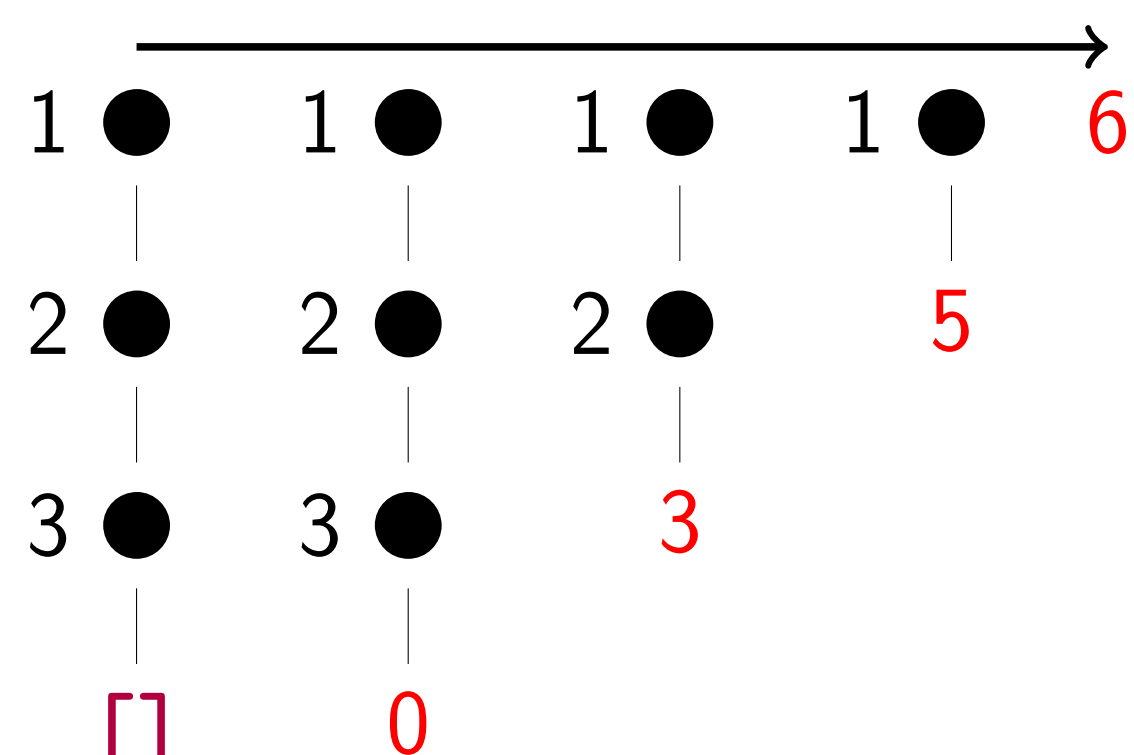
④ 計算効果への対応?

効率的なコードを生成するために, 計算効果を使いたい (可変状態, 継続)

- 既存研究 (Monadic Hylo) を流用 [Pardo2005]
- フュージョンによる効率向上が限定的 (≈ 1.15)

Banana $\llbracket \varphi \rrbracket = \text{Fold}$

```
sum :: [Int] -> Int
sum t = case t of
  [] -> 0
  h : t -> h + sum t
```



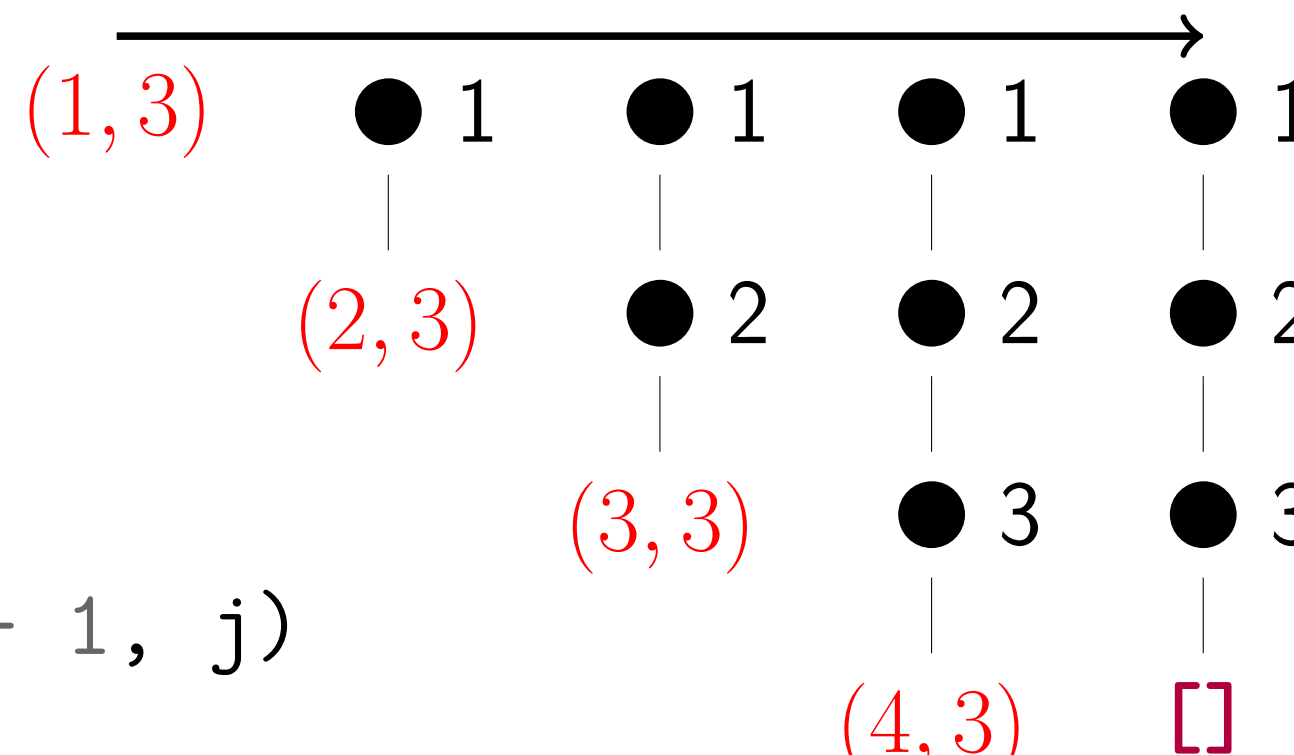
一般化したFold:

[Meijer1991]

$(\dashv) : (FB \rightarrow B) \rightarrow \mu F \rightarrow B$
 $\llbracket \varphi \rrbracket = \text{fix}(\lambda f. \varphi \circ F(f) \circ \text{out})$

Lens $\llbracket \psi \rrbracket = \text{Unfold}$

```
iota :: (Int, Int) -> [Int]
iota (i, j) =
  if i > j then []
  else i : iota (i + 1, j)
```



一般化したUnfold:

[Meijer1991]

$\llbracket - \rrbracket : (A \rightarrow FA) \rightarrow A \rightarrow \mu F$
 $\llbracket \psi \rrbracket = \text{fix}(\lambda f. \text{in} \circ F(f) \circ \psi)$

Envelope $\llbracket \varphi, \psi \rrbracket$

```
sum o iota
  || (Fusion)
h :: (Int, Int) -> Int
h (i, j) =
  if i > j then 0
  else i + h (i + 1, j)
```

再帰パターン: Hyломorphism [Meijer1991]

$\llbracket \varphi, \psi \rrbracket : A \rightarrow B$
 $\llbracket \varphi, \psi \rrbracket = \llbracket \varphi \rrbracket \circ \llbracket \psi \rrbracket$
 $= \text{fix}(\lambda f. \varphi \circ F(f) \circ \psi)$

例₁: 既存規則で対処可能な処理

F_1 が式を表現する場合, X は部分式に対応

$\tau : \forall X. (F_3 X \rightarrow X) \rightarrow F_2 X \rightarrow X$

[Takano1995]

$\llbracket \varphi, \eta_2, \text{out}_{F_3} \rrbracket_{F_4, F_3} \circ \llbracket \tau \text{ in}_{F_3}, \eta_1, \psi \rrbracket_{F_2, F_1} = \llbracket \tau(\varphi \circ \eta_2), \eta_1, \psi \rrbracket_{F_2, F_1}$

i.e., $\llbracket \varphi \circ \eta_2 \rrbracket$

フュージョン可能な処理: 部分式の中身に依存しないプログラム変換

$\text{let } x = M \text{ in } N \mapsto (\lambda x. N) M$
 $\tau : \forall x. (\text{Exp } v \ x \rightarrow x) \rightarrow \text{Exp } v \ x \rightarrow x$
 $\tau f (\text{Let } m \ h) = f(\text{App } f(\text{Lam } h) \ m)$

探索による最適化の高速化 (数論変換)

数論変換(NTT)とは, 有限体(mod p)上の高速フーリエ変換である.

■ 最適化₁: モジュロ演算を低レベルな演算に変換 (e.g., Barrett Reduction)

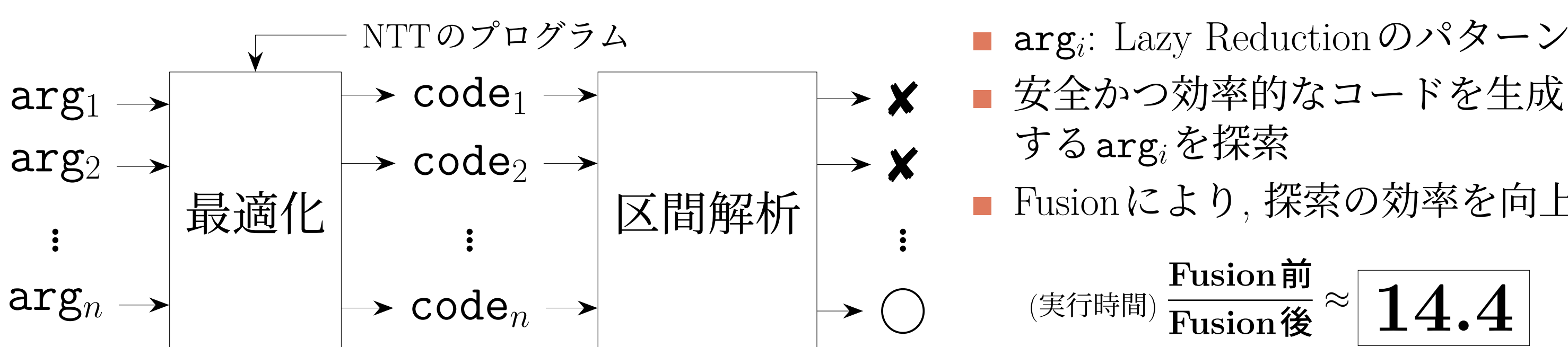
$x \bmod p \mapsto x - p * (x * 5 \gg 16)$

■ 最適化₂: 一部のモジュロ演算を省略 (a.k.a. Lazy Reduction)

$(x_1 + x_2 \bmod p) + x_3 \bmod p \mapsto (x_1 + x_2) + x_3 \bmod p$

モジュロ演算の回数 $\searrow \Rightarrow$ 効率 $\nearrow$ 整数overflowの可能性 $\nearrow$ 安全性 $\searrow$

■ 区間解析: 生成されるコードに対して静的に整数オーバーフローを検出



例₂: 拡張規則が必要となる処理

Gを用いて, 部分式の中身を解析可能

$\tau : \forall X. (F_3 X \rightarrow X) \rightarrow F_2 (GX) \rightarrow GX$

[本研究]

$G(\llbracket \varphi, \eta_2, \text{out}_{F_3} \rrbracket_{F_4, F_3}) \circ \llbracket \tau \text{ in}_{F_3}, \eta_1, \psi \rrbracket_{F_2, F_1} = \llbracket \tau(\varphi \circ \eta_2), \eta_1, \psi \rrbracket_{F_2, F_1}$

フュージョン可能な処理: Gでガイドしたプログラム変換

- G = $\langle \text{data Sign } x = \text{Pos } x \mid \text{Neg } x \rangle$: 部分式の符号を解析 (抽象解釈)
- G = $\langle \text{data P } x = \text{St Int} \mid \text{Dy } x \rangle$: 部分式の値を静的に評価 (部分評価)

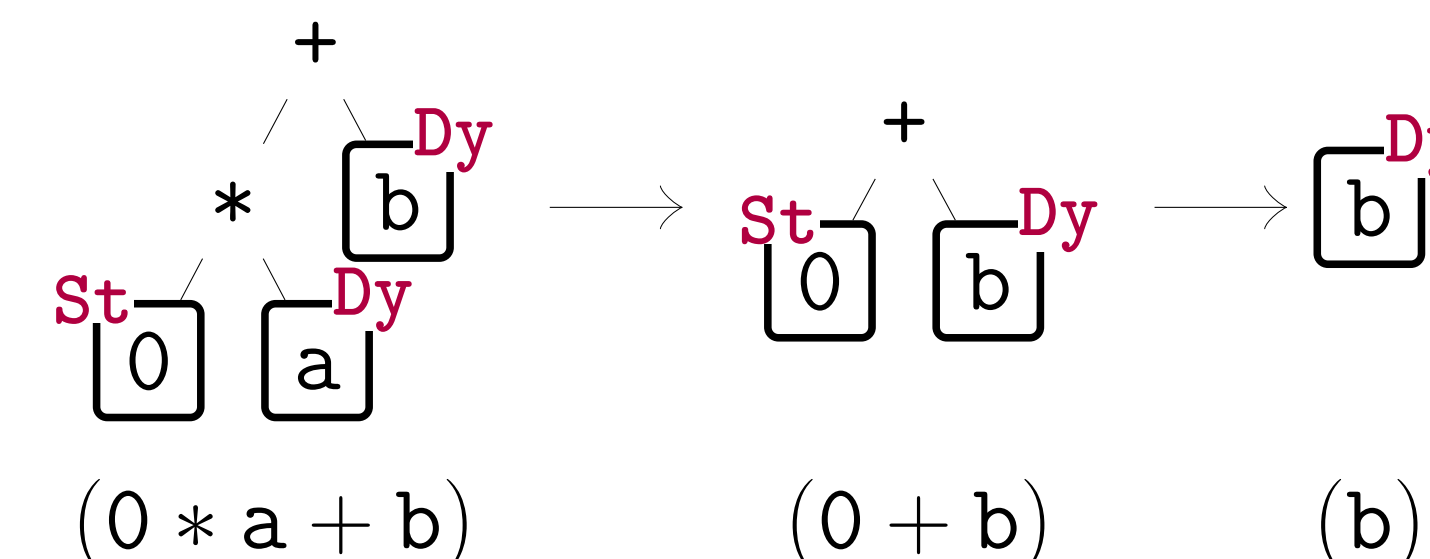
`data Exp x = Add x x | Mul x x | ...`

$\tau : \forall x. (\text{Exp } x \rightarrow x) \rightarrow \text{Exp } (P x) \rightarrow P x$

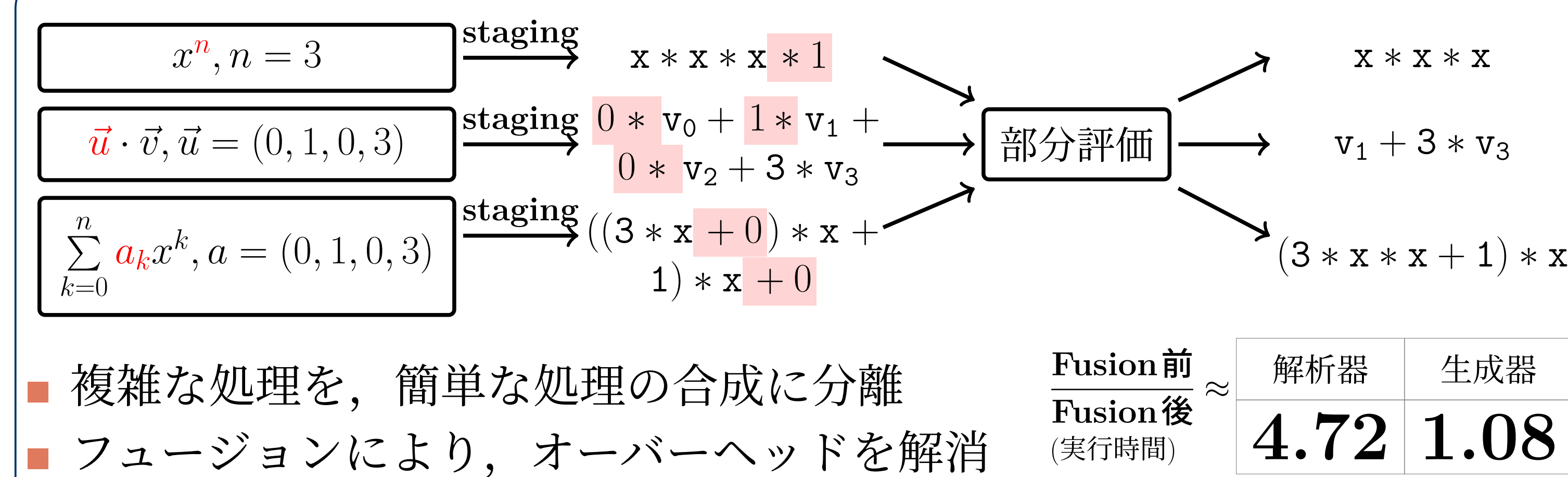
$\tau f (\text{Mul } (\text{St } 0) (\text{Dy } a)) = \text{St } 0$

$\tau f (\text{Add } (\text{St } 0) (\text{Dy } b)) = \text{Dy } b$

$\tau f (\text{Add } (\text{Dy } a) (\text{Dy } b)) = \text{Dy } (f(\text{Add } a \ b))$



ステージ化と部分評価の分離



計算効果への対応

効率的なコードを生成するために, 計算効果を持つHyломorphismを用いる

$\varphi : FB \rightarrow MB \quad \psi : A \rightarrow M(FA) \quad (M \text{ is a monad})$

- フュージョン規則はpureなHyloとeffectfulなHyloに対して定義 [Pardo2005]
- 予備実験の結果, モナドを導入した際のフュージョンによる効率向上は限定的だった (解析器における効率向上率 (pure/effectful) $\approx (9.41/1.15)$)

今後の課題

- PHOAS以外の束縛機構を持つ構造
- コード生成・解析においてUnfoldの応用
- リスクを伴う最適化の安全性保証 (aggressive & safe)